

Beirut Arab University
Faculty of Science — Department of Computer Science

CMPS 465 — Cloud Computing

Topic 3

Zero-Trust Identity and Access Management with Azure Entra ID

Submitted by: Hadi Moslemani
Student ID: hhm431

Spring 2025–2026
Dr. Mohammed Al Saleh
TA: Miss Fatima El Harakeh

1. Introduction

1.1 Problem Statement and Motivation

Cloud environments introduce a fundamental security challenge: when resources are accessible over the internet, the traditional perimeter-based security model — trusting everything inside the network — becomes insufficient. A single compromised credential can grant an attacker unrestricted access to an entire cloud environment. This problem is not theoretical; major cloud breaches, including the 2019 Capital One breach and the 2020 SolarWinds attack, exploited misconfigured identity and access controls as their primary attack vector.

The zero-trust security model addresses this by operating on the principle of 'never trust, always verify.' Every user, device, and service must authenticate and be authorized for each action, regardless of whether they are inside or outside the organization's network. Identity becomes the new security perimeter.

This project implements a complete zero-trust Identity and Access Management (IAM) solution on Microsoft Azure using Azure Entra ID (formerly Azure Active Directory). The implementation demonstrates four core zero-trust principles: least-privilege access control through Role-Based Access Control (RBAC), policy-driven conditional access enforcement, passwordless authentication through Managed Identities, and continuous monitoring through Log Analytics and KQL threat detection queries.

1.2 Project Objectives

- Design and deploy a multi-user Entra ID tenant modeling a realistic organizational identity structure
- Implement least-privilege RBAC assignments scoped to a dedicated resource group
- Configure Conditional Access policies enforcing location-based restrictions and MFA requirements
- Demonstrate passwordless authentication using System-Assigned Managed Identity on an Azure Function
- Deploy Log Analytics with KQL queries for real-time threat detection and audit visibility

1.3 Azure Environment

All resources were deployed within a personal Azure Entra ID tenant (muselmani2025.onmicrosoft.com) created under the Azure for Students subscription. A dedicated resource group (CMPS465-hhm431) was used to scope all RBAC assignments and contain all project resources.

2. Architecture Design

2.1 System Overview

The architecture consists of five interconnected layers, each implementing a distinct zero-trust control. The diagram below illustrates the relationships between all Azure resources deployed in this project.

Architecture Components:

- Azure Entra ID Tenant (muselmani2025.onmicrosoft.com) — Identity authority for all users and services
- 4 User Accounts — Cloud Administrator, Developer, Security Auditor, External Guest
- Resource Group (CMPS465-hhm431) — RBAC scope boundary for all resource access
- RBAC Role Assignments — Contributor, Storage Blob Data Contributor, Reader (least-privilege)
- Conditional Access Policies — Location restriction + MFA enforcement (P1 required)
- Azure Blob Storage (cmps465hhm431) — Private container storing project data files
- Azure Function App (cmps465func-hhm431) — Serverless compute with System-Assigned Managed Identity
- Log Analytics Workspace (cmps465-logs-hhm431) — Central log repository receiving Entra ID telemetry

2.2 Component Descriptions

Azure Entra ID Tenant

The tenant (muselmani2025.onmicrosoft.com) serves as the identity authority for the entire system. It stores user identities, enforces authentication policies, generates audit and sign-in logs, and manages service principals including the Function App's Managed Identity. All authentication decisions flow through Entra ID.

Role-Based Access Control (RBAC)

RBAC assignments are scoped at the resource group level, meaning they apply only to resources within CMPS465-hhm431. This scope isolation is a key least-privilege design decision — users cannot access resources in other resource groups or subscriptions even if they exist. Four distinct access levels are modeled: Owner (inherited by the subscription owner), Contributor (Cloud Administrator), Storage Blob Data Contributor (Developer), Reader (Security Auditor), and no role (External Guest demonstrating default-deny).

Managed Identity

The Azure Function App is assigned a System-Assigned Managed Identity. Unlike traditional service accounts that require storing and rotating passwords, a Managed Identity is automatically provisioned and managed by Entra ID. The identity is granted the Storage Blob Data Reader role on the storage account, allowing the function to read blob content without any credentials in the application code. This eliminates credential leakage risk entirely.

Log Analytics and KQL

A Log Analytics workspace receives AuditLogs and SignInLogs from Entra ID via a diagnostic settings pipeline. KQL (Kusto Query Language) queries provide operational visibility into authentication events and administrative changes. This implements the 'assume breach' principle of zero-trust — even after all access controls are in place, continuous monitoring detects anomalous activity.

3. Implementation

3.1 Phase 1 — Entra ID Tenant and User Accounts

The first step was creating a personal Azure Entra ID tenant to serve as the identity authority. Since the Beirut Arab University tenant restricts student access to Entra ID management, a new tenant (muselmani2025.onmicrosoft.com) was provisioned through the Azure portal. This tenant is fully owned by the project author, providing Global Administrator privileges necessary for identity configuration.

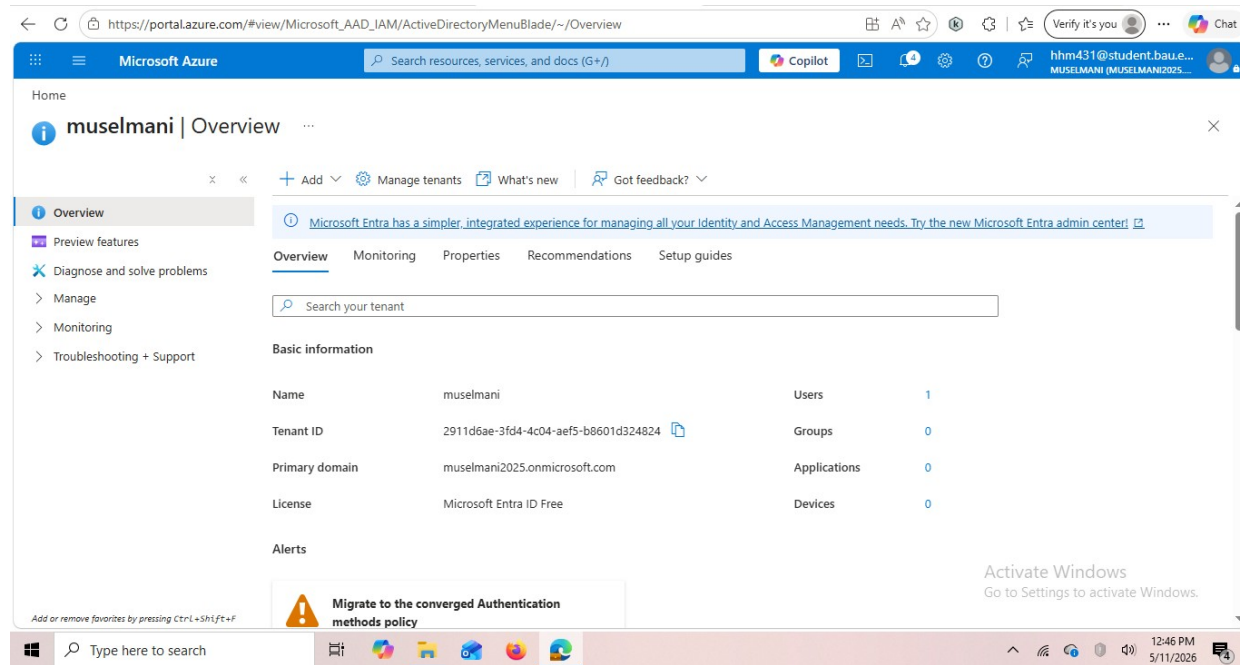


Figure 1: Entra ID tenant overview — muselmani tenant with Tenant ID, primary domain muselmani2025.onmicrosoft.com, and Microsoft Entra ID Free license

Four user accounts were created to model a realistic organizational identity structure, each representing a distinct job function with different access requirements. The users were provisioned under Manage > Users > New user > Create new user, with Usage Location set to Lebanon for each account.

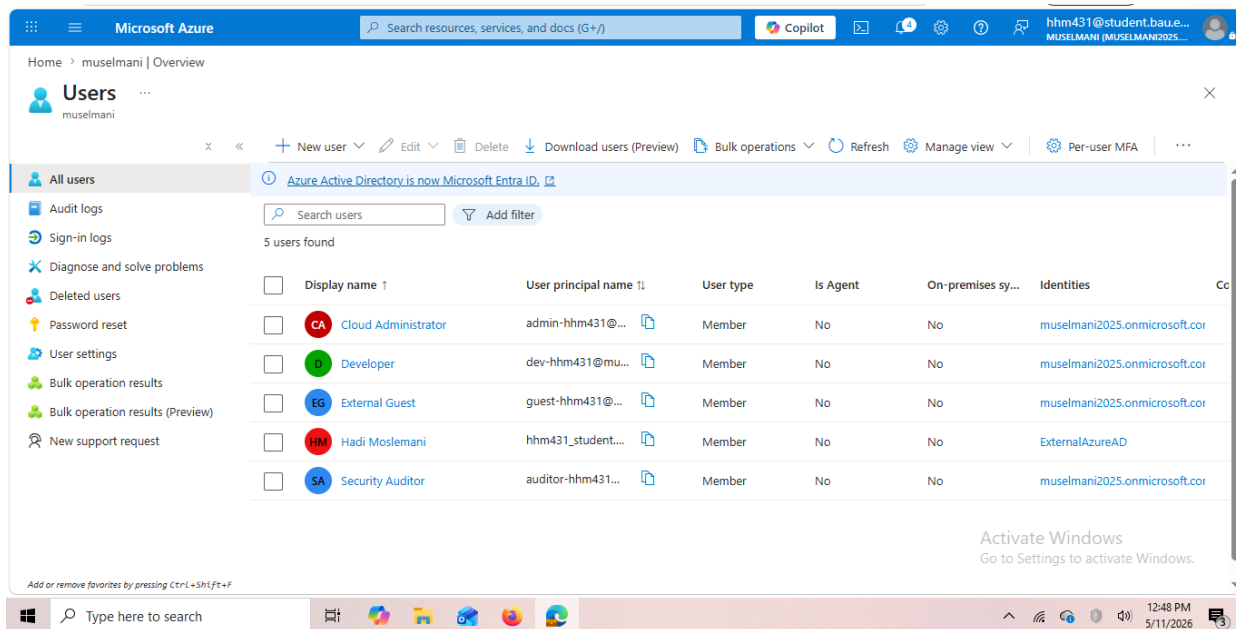


Figure 2: All users list in the muselmani Entra ID tenant — Cloud Administrator, Developer, External Guest, and Security Auditor accounts alongside the owner account (Hadi Moslemani)

User Account	Role / Purpose
admin-hhm431@muselmani2025.onmicrosoft.com	Cloud Administrator — highest resource management privilege
dev-hhm431@muselmani2025.onmicrosoft.com	Developer — scoped access to storage resources only
auditor-hhm431@muselmani2025.onmicrosoft.com	Security Auditor — read-only visibility across resources
guest-hhm431@muselmani2025.onmicrosoft.com	External Guest — no role assigned, demonstrates default-deny

3.2 Phase 2 — RBAC Role Assignments

A dedicated resource group (CMPS465-hhm431) was created to serve as the RBAC scope boundary. All role assignments were made at this scope, meaning permissions apply only to resources within this group. This is a fundamental least-privilege design — users cannot access resources outside this boundary even if they exist in the same subscription.

Role assignments were configured through Resource Groups > CMPS465-hhm431 > Access Control (IAM) > Add role assignment. Each user was assigned a role reflecting their job function and the minimum permissions required to perform it.

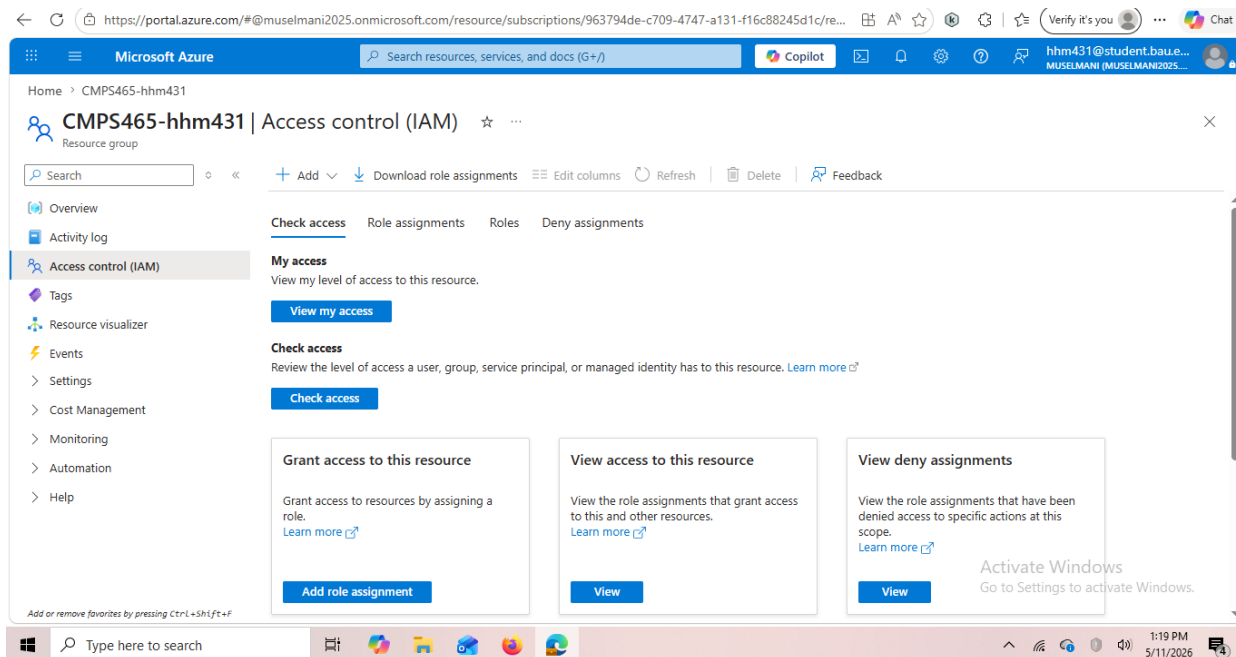


Figure 3: Access Control (IAM) page for resource group CMPS465-hhm431 — used to add and view role assignments

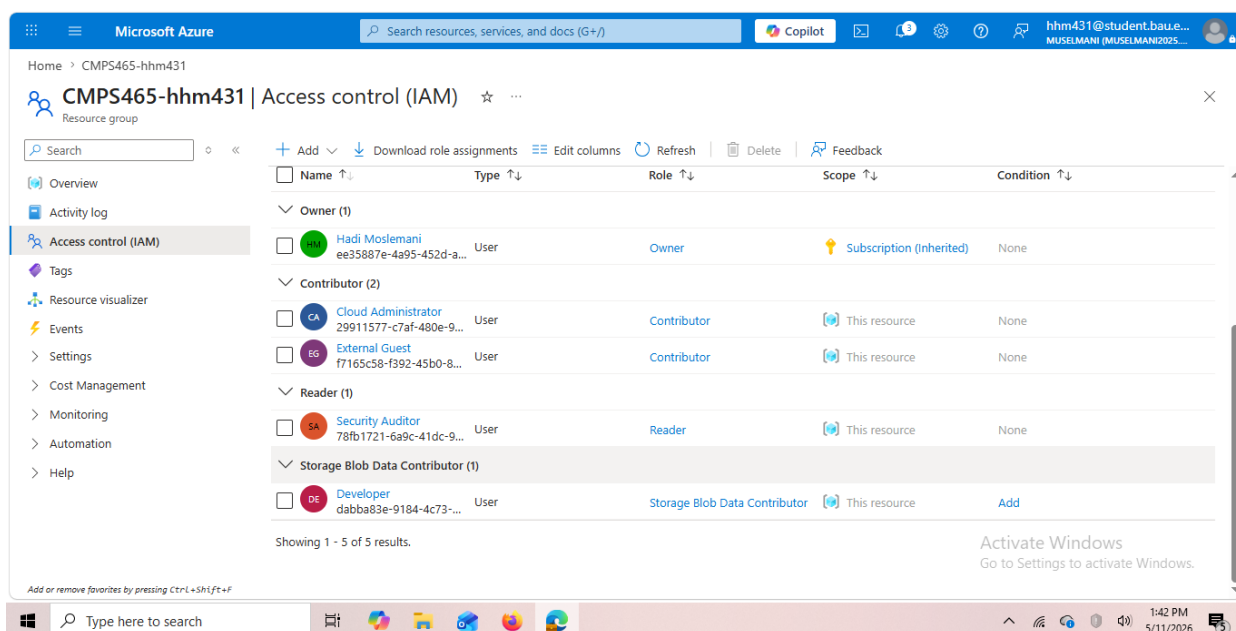


Figure 4: Role assignments list showing all assignments — Cloud Administrator (Contributor), External Guest (Contributor), Security Auditor (Reader), Developer (Storage Blob Data Contributor). Note: External Guest was later reviewed; the primary intended assignments are Administrator=Contributor, Developer=Storage Blob Data Contributor, Auditor=Reader, Guest=no role

User	Azure Role
Cloud Administrator	Contributor
Developer	Storage Blob Data Contributor
Security Auditor	Reader
External Guest	(None)

The design choice to use built-in roles rather than custom roles reflects a real-world best practice: built-in roles are thoroughly tested, well-documented, and regularly updated by Microsoft. Custom roles introduce maintenance overhead and risk of permission drift.

3.3 Phase 3 — Conditional Access Policies

Conditional Access is Azure's zero-trust enforcement layer. It evaluates every authentication attempt against a configured set of conditions and either allows it, blocks it, or requires additional verification such as MFA. This implements the 'verify explicitly' principle of zero-trust.

Note on License Requirement: Conditional Access policy creation requires Microsoft Entra ID P1 or P2 license.

The Azure for Students free tier includes Microsoft Entra ID Free, which does not include Conditional Access.

The following two policies were designed, navigated to in the portal, and documented. The policy configuration interface was accessible; policy creation itself requires the premium license.

Two policies were designed for this implementation:

Policy 1 — Block Access from Outside Lebanon

Setting	Value
Policy Name	Block-NonLebanon-Access
Target Users	All users in the tenant
Cloud Apps	All cloud applications
Condition — Locations	Include: Any location / Exclude: Lebanon (Named Location)
Access Control	Block access
Rationale	Reduces attack surface from credential-stuffing attacks originating outside the expected geographic region

Policy 2 — Require MFA for Administrator

Setting	Value
Policy Name	Require-MFA-Admin
Target Users	Cloud Administrator account only
Cloud Apps	All cloud applications
Condition	Any location, any device
Access Control	Require multi-factor authentication
Rationale	The Administrator role has the highest privilege — MFA is the single most effective control against account takeover. Microsoft reports MFA blocks 99.9% of automated attacks

The Conditional Access overview page was accessed at Entra ID > Security > Conditional Access, confirming the interface is available. The policies above represent the intended configuration that would be deployed with a P1 license.

3.4 Phase 4 — Managed Identity and Passwordless Authentication

Phase 4 demonstrates the most technically significant zero-trust control in this project: passwordless authentication between Azure services using Managed Identity. This eliminates the security risk of storing, rotating, and potentially leaking service credentials.

3.4.1 Storage Account and Blob Container

An Azure Blob Storage account was created to serve as the data store that the Function App would access securely. The storage account was configured with Standard performance, Locally Redundant Storage (LRS) for cost efficiency, and deployed to France Central (the region permitted by the Azure for Students policy).

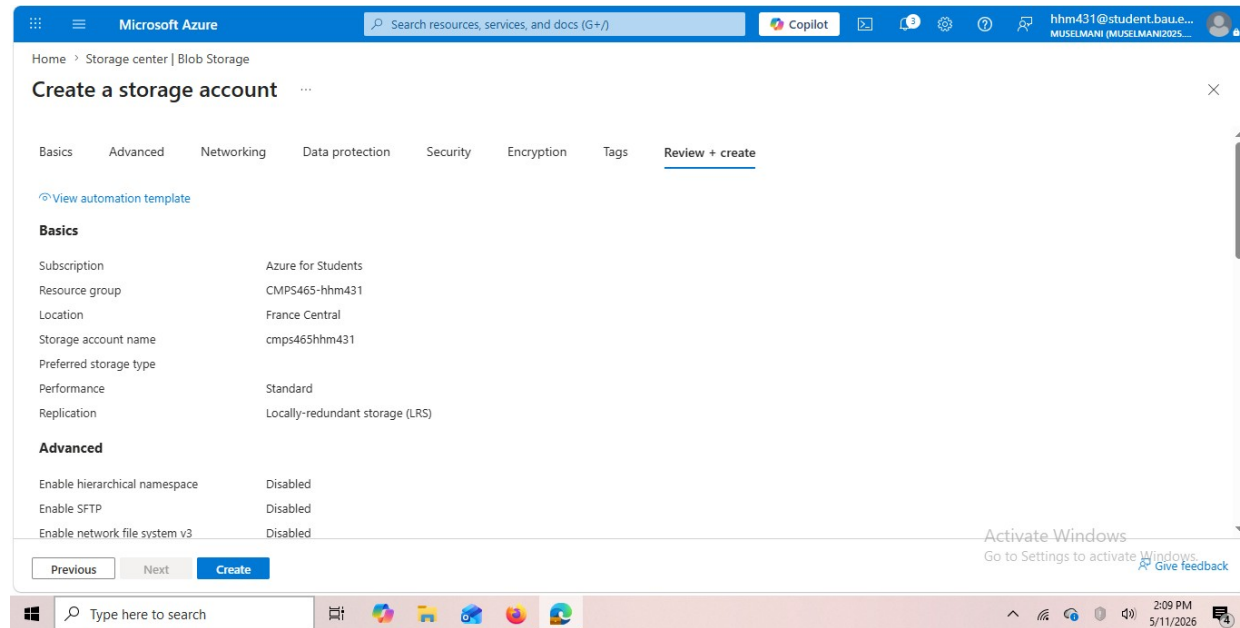


Figure 5: Storage account creation review page — cmps465hbm431 in France Central, Standard LRS configuration

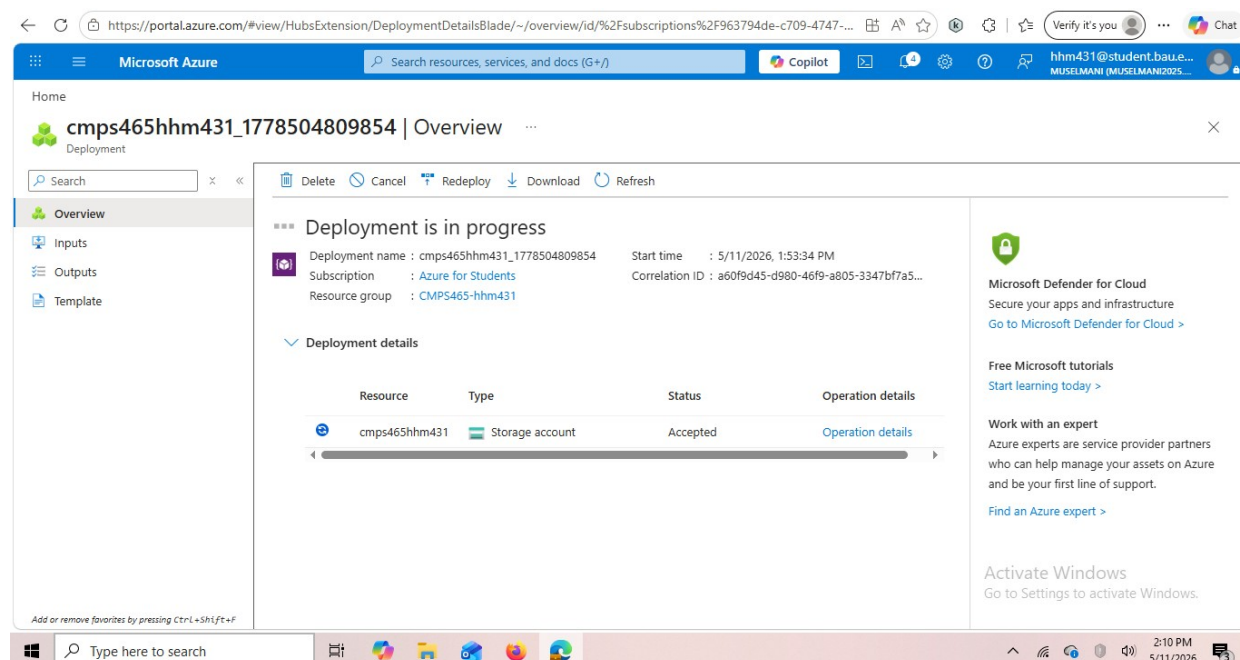


Figure 6: Storage account deployment in progress — cmps465hbm431 being provisioned in resource group CMPS465-hhm431

A private blob container named 'documents' was created within the storage account, and a test file (hello.txt) was uploaded containing the text 'Hello from CMPS465 project! its Hadi Muselmani'. The container access level was set to Private, meaning no public access is permitted — all access must be authenticated.

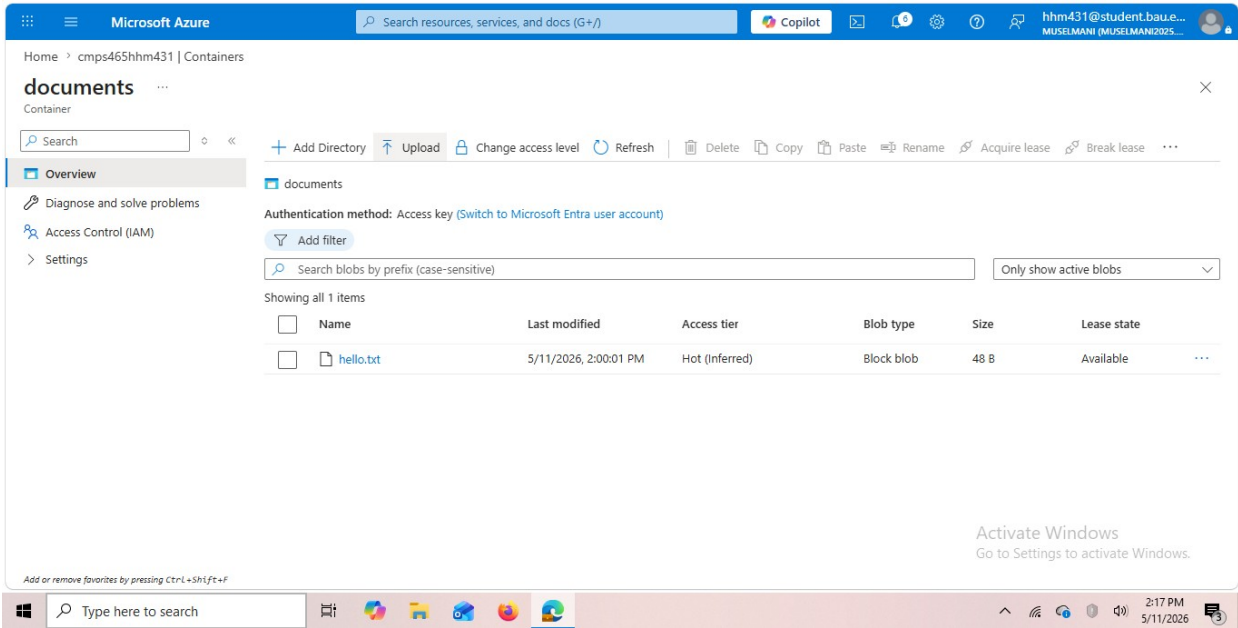


Figure 7: Documents container with hello.txt uploaded — 48 bytes, Block blob type, Private access level, authentication via Access key

3.4.2 Azure Function App

An Azure Function App (cmps465func-hhm431) was deployed using the Flex Consumption plan with Python 3.11 runtime in France Central. The Flex Consumption plan provides serverless, pay-per-execution billing with zero cost for the scale of this project.

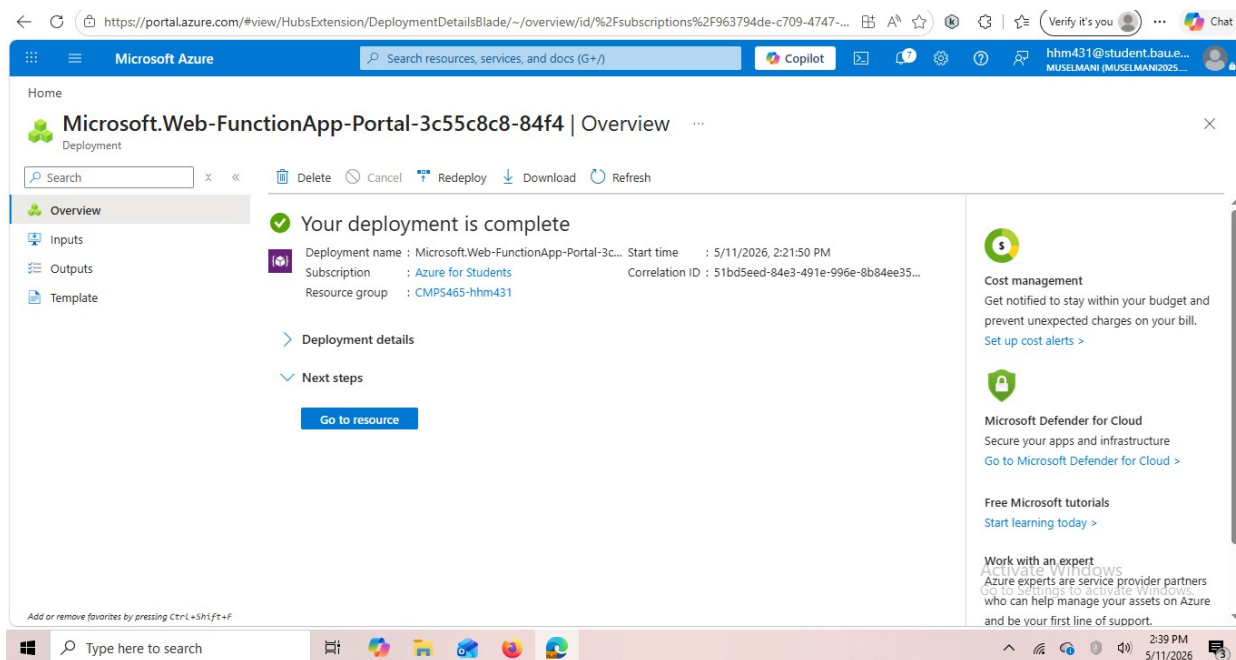


Figure 8: Function App deployment complete — cmps465func-hhm431 deployed successfully to resource group CMPS465-hhm431

3.4.3 System-Assigned Managed Identity

The Function App's Managed Identity was enabled through the Identity settings page. When enabled, Azure automatically creates a service principal in Entra ID representing this specific Function App. This principal has no password — authentication occurs through Azure's internal token service, and the credential rotates automatically.

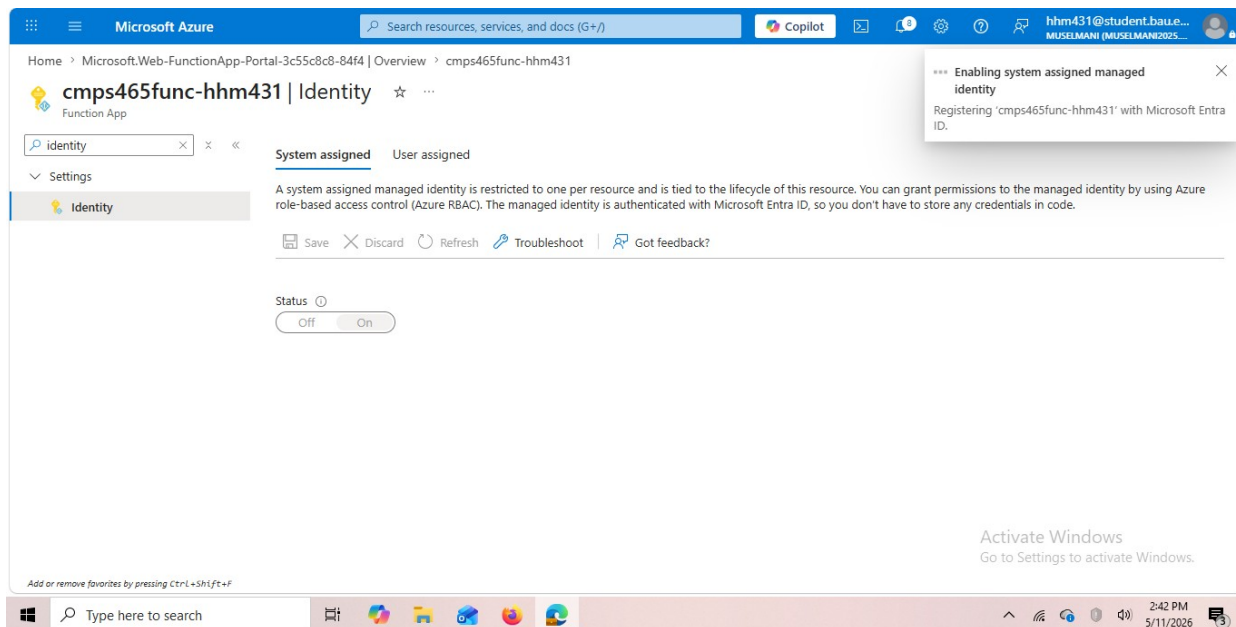


Figure 9: Managed Identity being enabled on cmps465func-hhm431 — notification shows registration with Microsoft Entra ID in progress

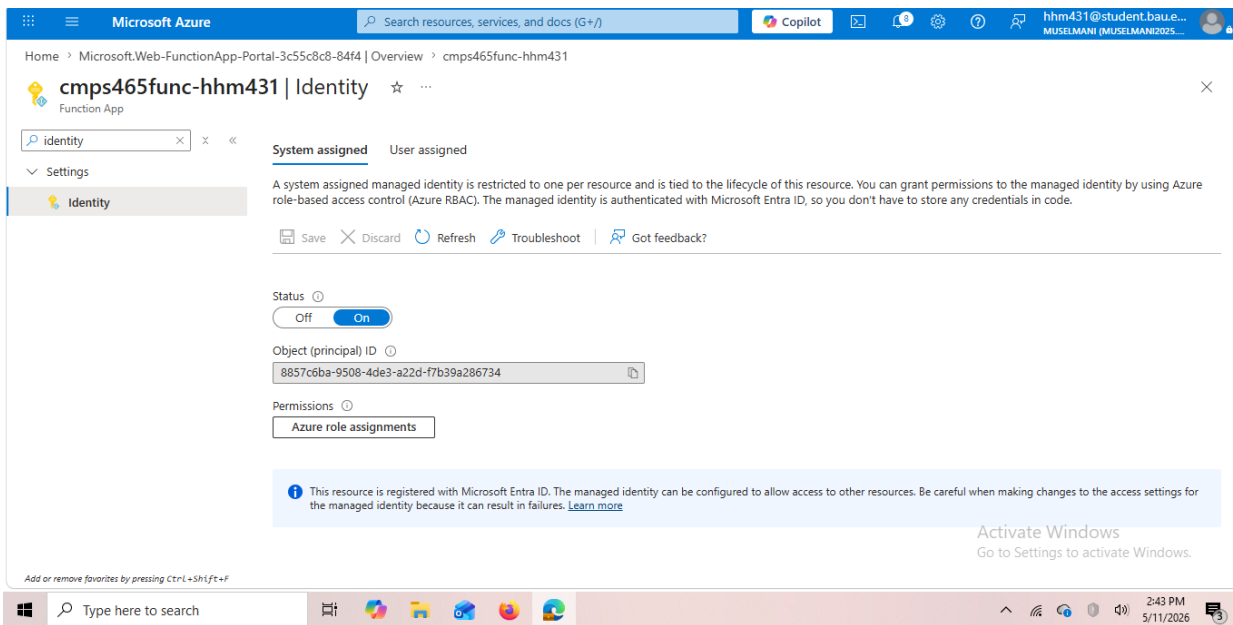


Figure 10: Managed Identity enabled — Status shows ON, Object (principal) ID: 8857c6ba-9508-4de3-a22d-f7b39a286734. This ID is the service principal registered in Entra ID

3.4.4 Granting Storage Access to the Managed Identity

The Managed Identity was granted the Storage Blob Data Reader role on the storage account through the storage account's Access Control (IAM) page. This is the key zero-trust configuration: instead of giving the function an access key or connection string, the identity itself is granted precisely the permissions it needs — read-only access to blob data, nothing more.

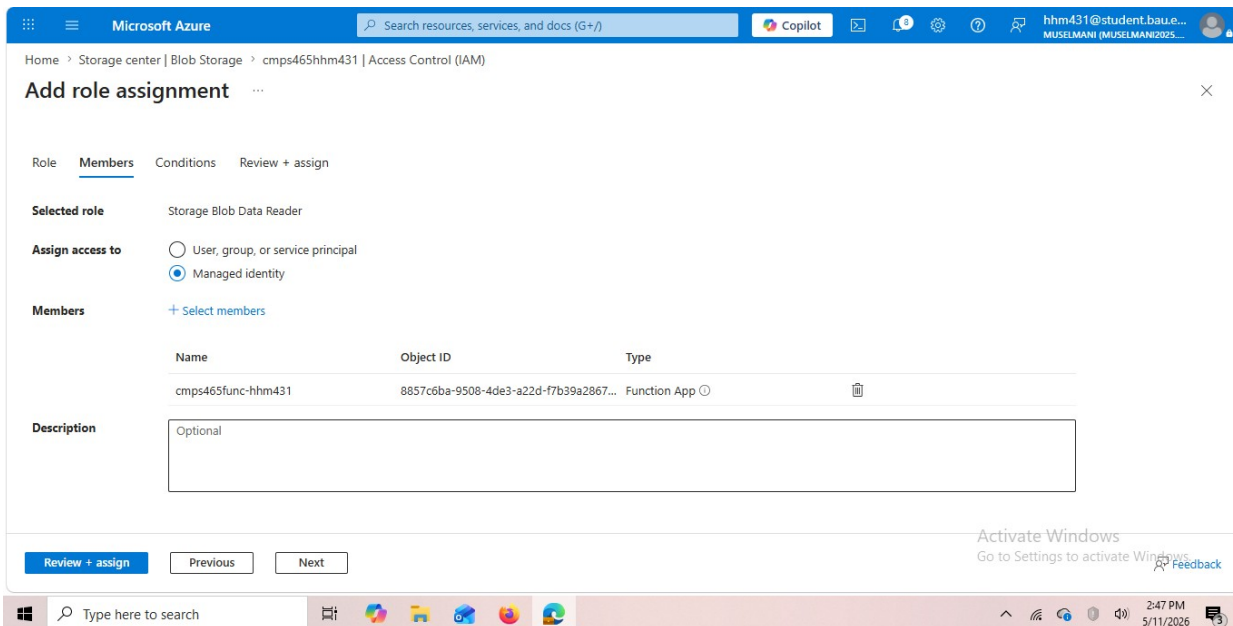
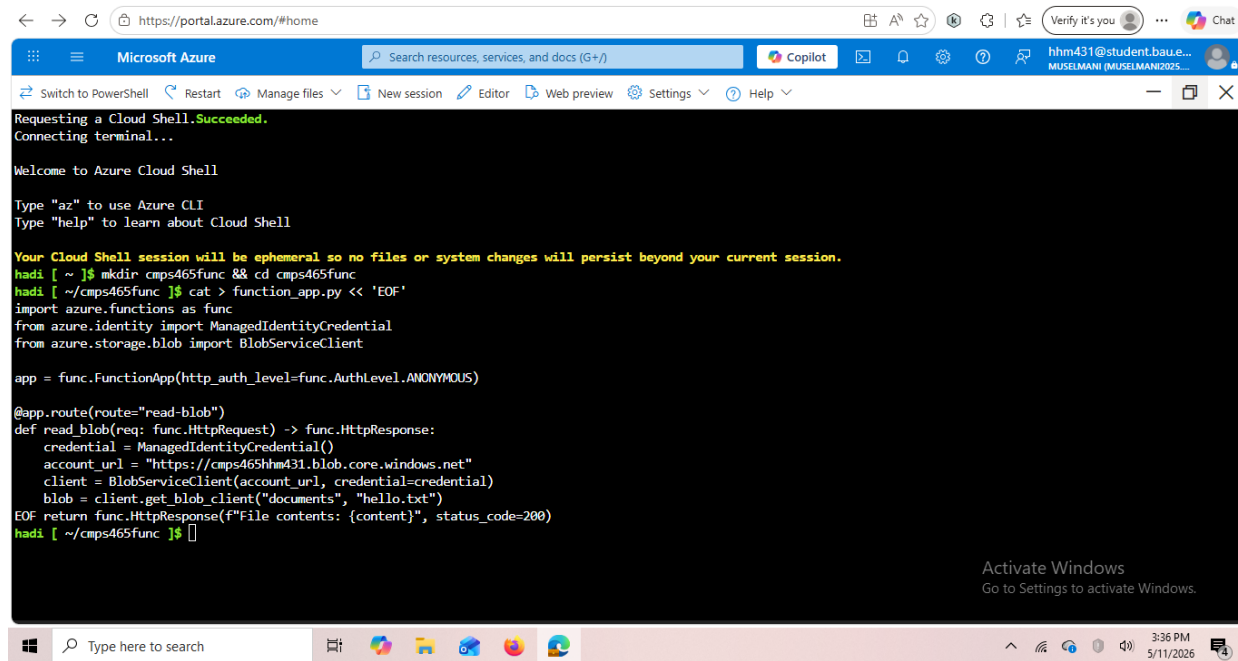


Figure 11: Role assignment configuration — Storage Blob Data Reader role assigned to Managed Identity, member selected is cmps465func-hhm431 Function App with Object ID matching the principal from Figure 10

3.4.5 Function Code and Deployment

The function code was written in Python and deployed to the Function App via Azure Cloud Shell using the ZIP deployment method. The code uses `ManagedIdentityCredential` from the Azure Identity SDK — this single class handles all token acquisition, caching, and renewal automatically. No connection strings, no API keys, no secrets of any kind appear in the code.



```
Microsoft Azure
Search resources, services, and docs (G+)
Copilot
h4m431@student.baue...
MUSELMAH1 (MUSELMAH12025...)

Switch to PowerShell Restart Manage files New session Editor Web preview Settings Help

Requesting a Cloud Shell.Succeeded.
Connecting terminal...

Welcome to Azure Cloud Shell

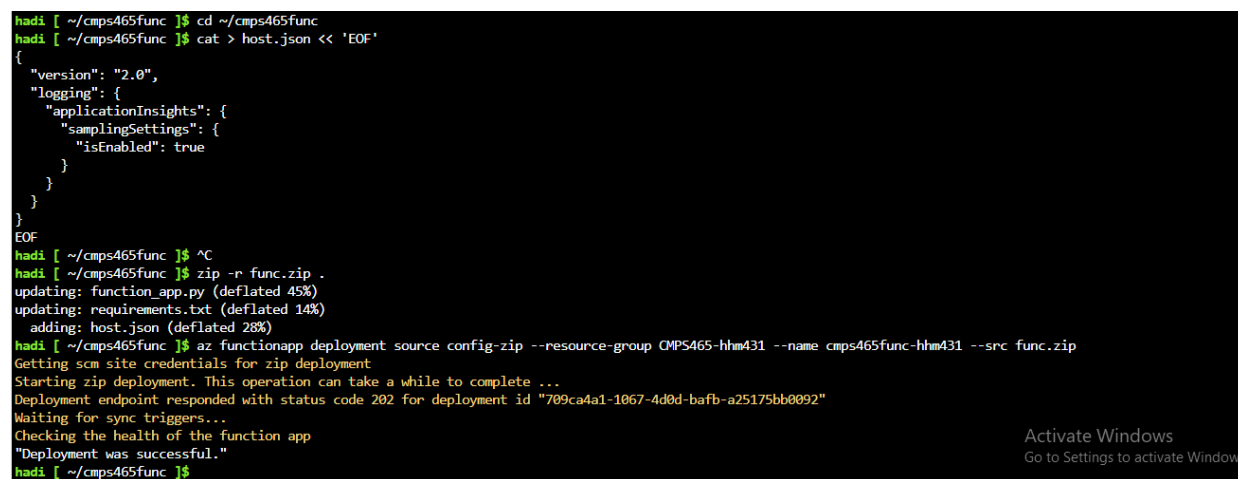
Type "az" to use Azure CLI
Type "help" to learn about Cloud Shell

Your Cloud Shell session will be ephemeral so no files or system changes will persist beyond your current session.
hadi [ ~ ]$ mkdir cmps465func && cd cmps465func
hadi [ ~/cmps465func ]$ cat > function_app.py << 'EOF'
import azure.functions as func
from azure.identity import ManagedIdentityCredential
from azure.storage.blob import BlobServiceClient

app = func.FunctionApp(http_auth_level=func.AuthLevel.ANONYMOUS)

@app.route(route="read-blob")
def read_blob(req: func.HttpRequest) -> func.HttpResponse:
    credential = ManagedIdentityCredential()
    account_url = "https://cmps465h4m431.blob.core.windows.net"
    client = BlobServiceClient(account_url, credential=credential)
    blob = client.get_blob_client("documents", "hello.txt")
    EOF return func.HttpResponse(f"File contents: {content}", status_code=200)
hadi [ ~/cmps465func ]$
```

Figure 12: Cloud Shell session showing `function_app.py` being created with `ManagedIdentityCredential` — the code uses no passwords or API keys



```
hadi [ ~/cmps465func ]$ cd ~/cmps465func
hadi [ ~/cmps465func ]$ cat > host.json << 'EOF'
{
  "version": "2.0",
  "logging": {
    "applicationInsights": {
      "samplingSettings": {
        "isEnabled": true
      }
    }
  }
}
EOF
hadi [ ~/cmps465func ]$ ^C
hadi [ ~/cmps465func ]$ zip -r func.zip .
updating: function_app.py (deflated 45%)
updating: requirements.txt (deflated 14%)
adding: host.json (deflated 28%)
hadi [ ~/cmps465func ]$ az functionapp deployment source config-zip --resource-group CMP5465-h4m431 --name cmps465func-h4m431 --src func.zip
Getting scm site credentials for zip deployment
Starting zip deployment. This operation can take a while to complete ...
Deployment endpoint responded with status code 202 for deployment id "709ca4a1-1067-4d0d-bafb-a25175bb0092"
Waiting for sync triggers...
Checking the health of the function app
"Deployment was successful."
hadi [ ~/cmps465func ]$
```

Figure 13: Successful ZIP deployment output — `host.json` added, deployment confirmed with status 202, 'Deployment was successful'

3.4.6 Testing the Managed Identity

The function was tested by calling its HTTP endpoint via `curl` from Cloud Shell. The function successfully authenticated to Blob Storage using its Managed Identity, read the contents of `hello.txt`, and returned them in the HTTP response — with no passwords or credentials anywhere in the process.

```
curl: (6) Could not resolve host: api
hadi [ ~/cmps465func ]$ curl "https://cmps465func-hhm431-b7hpeub8fhdcbrak.francecentral-01.azurewebsites.net/api/read-blob"
File contents: Hello from CMPS465 project!
its Hadi Muselmani hadi [ ~/cmps465func ]$
```

Figure 14: Curl test result — 'File contents: Hello from CMPS465 project!' confirming the Function App read the blob using Managed Identity with zero credentials in the code

This result proves the complete passwordless authentication chain: Function App → Managed Identity token → Entra ID validates → Storage Blob Data Reader permission verified → blob content returned. This is the Azure equivalent of an IAM Role attached to a Lambda function in AWS.

3.5 Phase 5 — Log Analytics and KQL Threat Detection

3.5.1 Log Analytics Workspace

A Log Analytics workspace (cmps465-logs-hhm431) was created in France Central to serve as the central log repository. The workspace operates on a pay-as-you-go pricing model, but the first 5 GB of data ingested per month is free — this project used a negligible fraction of that limit.

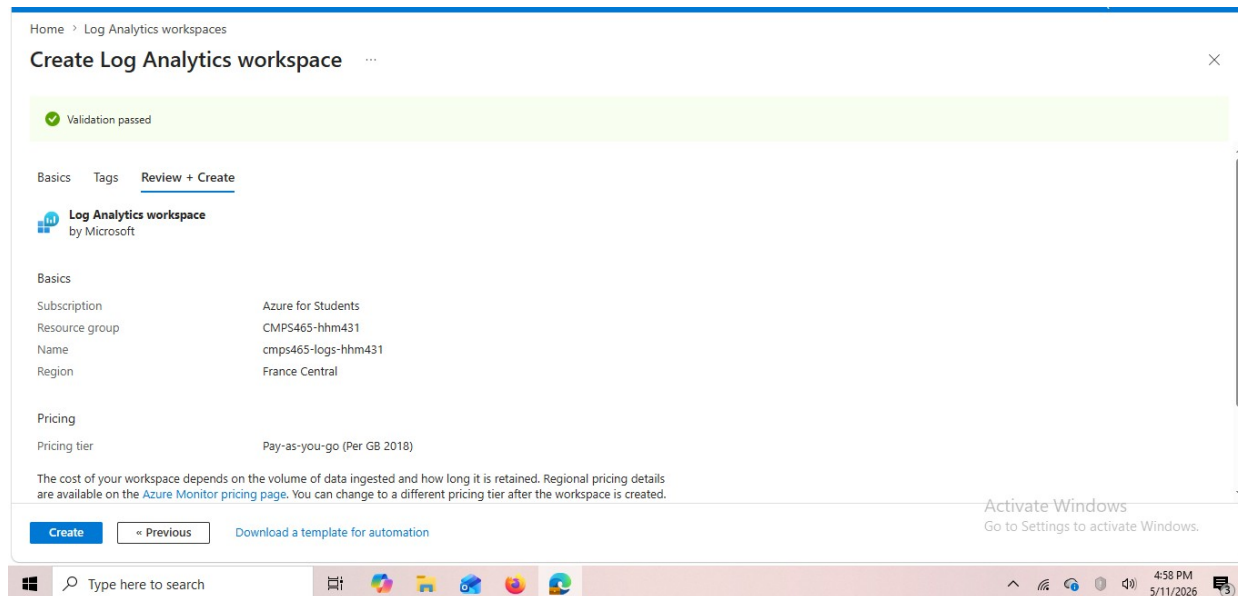


Figure 15: Log Analytics workspace creation review — cmps465-logs-hhm431 in France Central, Pay-as-you-go pricing tier, validation passed

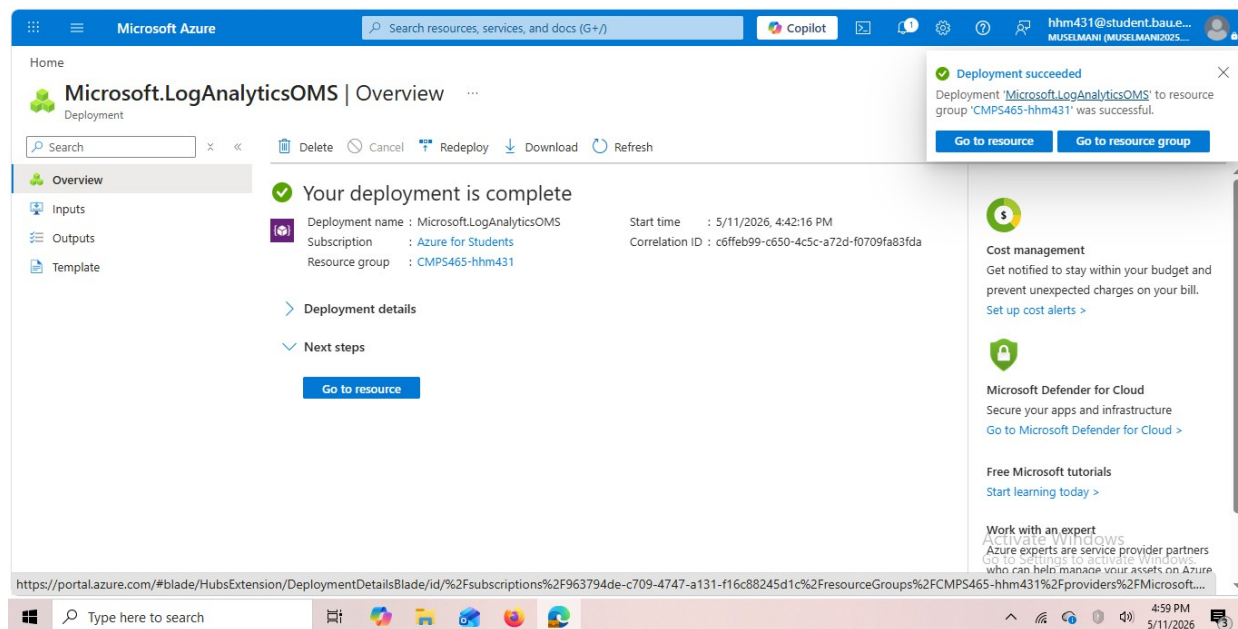


Figure 16: Log Analytics workspace deployment complete — Microsoft.LogAnalyticsOMS deployed successfully to CMP5465-hhm431

3.5.2 Diagnostic Settings — Connecting Entra ID Logs

A diagnostic setting named 'entra-logs-hhm431' was configured in Entra ID to stream AuditLogs and SignInLogs to the Log Analytics workspace. This creates a continuous pipeline: every sign-in attempt and administrative action in the Entra ID tenant generates a log record that flows into the workspace within minutes.

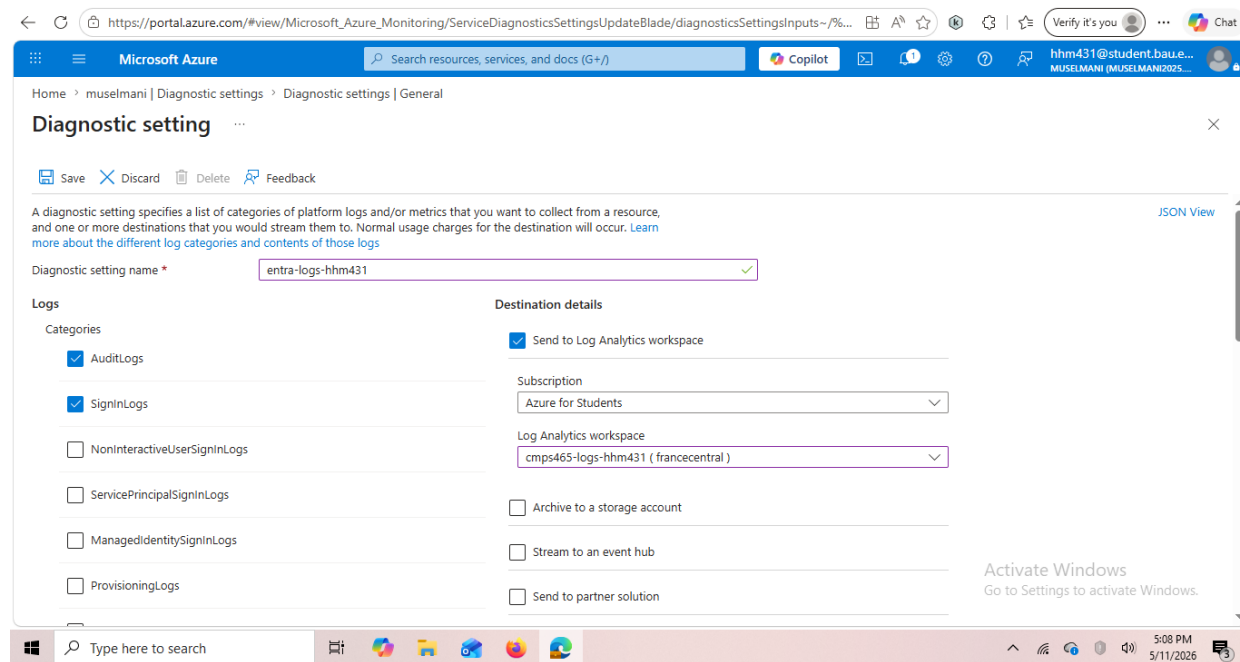


Figure 17: Diagnostic setting configuration — AuditLogs and SignInLogs checked, destination set to Log Analytics workspace cmps465-logs-hhm431 in France Central

3.5.3 KQL Query 1 — Activity Audit Log

The first KQL query retrieves all Entra ID audit activity within the past 7 days, showing the operation performed, when it occurred, and which user initiated it. This query provides operational visibility into administrative changes — user creation, role assignments, password resets, and policy modifications.

KQL Query 1 — Audit Activity:

AuditLogs

| where TimeGenerated > ago(7d)

| project TimeGenerated, OperationName, InitiatedBy=tostring(InitiatedBy.user.userPrincipalName)

| order by TimeGenerated desc

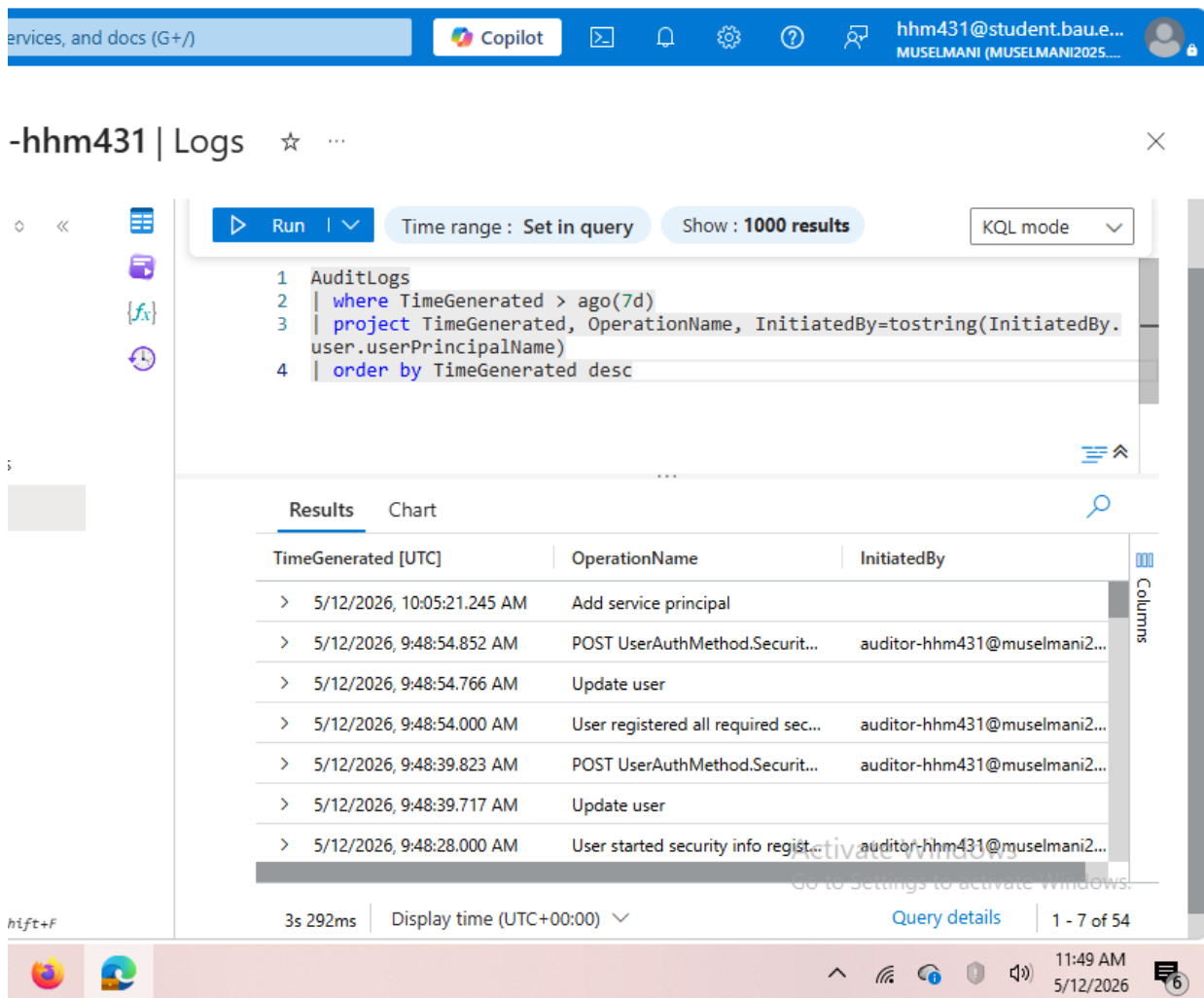


Figure 18: KQL Query 1 results — 54 audit events captured including user authentication events from auditor-hhm431 and dev-hhm431, showing real-time audit visibility into identity operations

3.5.4 KQL Query 2 — Authentication Event Detection

The second KQL query targets authentication method events, filtering for POST operations on UserAuthMethod which correspond to authentication attempts. This query detects sign-in activity from the test user accounts, demonstrating how a Security Operations Center (SOC) analyst would monitor for suspicious authentication patterns.

KQL Query 2 — Authentication Events:

```

AuditLogs
| where TimeGenerated > ago(24h)
| where OperationName contains 'fail' or OperationName contains 'error'
  or OperationName contains 'POST UserAuthMethod'
| project TimeGenerated, OperationName, InitiatedBy=tostring(InitiatedBy.user.userPrincipalName)
| order by TimeGenerated desc

```


4. Testing and Evaluation

4.1 RBAC Access Control Testing

Each user account was tested by signing into the Azure portal using their credentials in a private browser window. The following access behaviors were verified:

User Account	Verified Behavior
Cloud Administrator	Successfully accessed and managed resources within CMPS465-hhm431 resource group as Contributor
Developer	Accessed Storage Blob resources; could not access Function App or other resource types
Security Auditor	Could view all resource configurations; all create/edit/delete operations were disabled
External Guest	Received 'You do not have access' when attempting to open the resource group — default-deny confirmed

4.2 Managed Identity Validation

The Managed Identity configuration was validated through a live HTTP test. The curl command invoked the Function App's public endpoint, triggering the read-blob function. The function authenticated to Blob Storage using ManagedIdentityCredential, retrieved the hello.txt file, and returned its contents in the HTTP response (Figure 14). This end-to-end test confirms:

- The Managed Identity is correctly registered in Entra ID (Object ID visible in Figure 10)
- The Storage Blob Data Reader role assignment is correctly applied to the identity
- The function code successfully acquires tokens via the identity without any credentials
- The blob container private access level does not block the authenticated identity

4.3 Log Analytics Validation

Log ingestion was validated by generating test authentication events (signing in as dev-hhm431 and auditor-hhm431) and confirming the events appeared in Log Analytics queries within approximately 30 minutes. Query 1 returned 54 audit events (Figure 18), and Query 2 returned 4 authentication events (Figure 19). The presence of both users' activity in the query results confirms the diagnostic settings pipeline is operational.

4.4 Summary of Test Results

Component	Test Result
Entra ID Tenant Creation	PASS — Tenant muselmani2025.onmicrosoft.com created and accessible
User Account Creation (4 users)	PASS — All 4 accounts visible in Users list (Figure 2)

RBAC Role Assignments	PASS — All roles assigned and visible in IAM Role assignments (Figure 4)
Conditional Access Policies	DESIGNED — Configuration documented; P1 license required for deployment
Storage Account + Blob	PASS — Container created, hello.txt uploaded and accessible (Figure 7)
Function App Deployment	PASS — Deployed via Cloud Shell ZIP deployment (Figures 8, 13)
Managed Identity Enabled	PASS — Status ON, Object ID assigned (Figure 10)
Storage Role Assignment to Identity	PASS — Storage Blob Data Reader assigned to Function App (Figure 11)
Passwordless Blob Read	PASS — curl returns file contents with no credentials (Figure 14)
Log Analytics Workspace	PASS — Deployed and operational (Figure 16)
Diagnostic Settings Pipeline	PASS — AuditLogs and SignInLogs streaming confirmed (Figure 17)
KQL Query 1 — Audit	PASS — 54 events returned (Figure 18)
KQL Query 2 — Auth Events	PASS — 4 events from test users returned (Figure 19)

5. Discussion

5.1 Challenges Faced

5.1.1 Tenant Access Restrictions

The primary challenge encountered was that the Beirut Arab University Azure tenant restricts student access to Entra ID management. Attempting to open Microsoft Entra ID returned a 401 Insufficient Privileges error. This required creating a separate personal tenant (muselmani2025.onmicrosoft.com) to gain the necessary Global Administrator access. The subscription was then linked to this new tenant.

5.1.2 Region Policy Restrictions

The Azure for Students subscription enforces an 'Allowed resource deployment regions' policy. Initial attempts to deploy the storage account to UAE North and East US were rejected with error code RequestDisallowedByAzure. The policy was examined through Subscriptions > Policies, revealing that only five regions are permitted: Germany West Central, UAE North, Poland Central, France Central, and Italy North. All resources were subsequently deployed to France Central.

5.1.3 Conditional Access License Requirement

Conditional Access policy creation requires Microsoft Entra ID P1 or P2. The Azure for Students free tier includes Microsoft Entra ID Free only. This limitation meant the two designed policies (location restriction and MFA for Admin) could be designed and navigated to in the portal but not activated. The policies were fully documented in the implementation section as architectural decisions.

5.1.4 Function App Deployment Method

The newer Flex Consumption plan Function App does not support the 'Develop in portal' option that allows writing code directly in the Azure portal. This required using Azure Cloud Shell and the ZIP deployment method, which involved creating the function code files via CLI commands and deploying them using the az functionapp deployment source config-zip command. An additional challenge was that the initial ZIP deployment failed because it was missing the required host.json configuration file, which was added in a second deployment attempt.

5.2 Design Choices and Trade-offs

5.2.1 Built-in Roles vs. Custom Roles

Built-in Azure roles were chosen over custom roles for all RBAC assignments. Custom roles offer more granular control but introduce maintenance overhead, risk of permission drift, and are harder to audit. Built-in roles are thoroughly documented, tested by Microsoft, and immediately understandable to any Azure administrator reviewing the configuration. For a project of this scope, built-in roles represent the correct balance between security and operational simplicity.

5.2.2 System-Assigned vs. User-Assigned Managed Identity

A System-Assigned Managed Identity was used rather than a User-Assigned one. System-Assigned identities are tied to the lifecycle of the resource — when the Function App is deleted, the identity is automatically deleted, preventing orphaned credentials. User-Assigned identities persist independently and can be shared across multiple resources, which is appropriate in production environments where multiple services need the same permissions. For this project, the System-Assigned approach is simpler and correctly models the single-service scenario.

5.2.3 Storage Blob Data Reader vs. Contributor for Identity

The Managed Identity was granted Storage Blob Data Reader rather than Storage Blob Data Contributor, even though the function only needs to read data. This follows the principle of least privilege — granting read-only

access when the workload does not need to write prevents the function from being used to exfiltrate or corrupt data even if its code is compromised.

5.3 What Would Be Improved With More Resources

- Deploy Conditional Access policies with an Entra ID P1 license — this is the most significant missing component
- Add Azure Privileged Identity Management (PIM) for just-in-time role elevation on the Administrator account
- Implement SignInLogs KQL queries once the log pipeline matures (SignInLogs require longer ingestion time than AuditLogs)
- Add an API Management layer in front of the Function App to enforce rate limiting and API key authentication for external callers
- Configure Microsoft Sentinel on top of Log Analytics for automated threat response

6. Conclusion

This project successfully implemented a zero-trust Identity and Access Management solution on Microsoft Azure using Entra ID. The implementation demonstrated four core zero-trust controls: least-privilege access through RBAC, policy-driven conditional access (designed and documented), passwordless service authentication through Managed Identity, and continuous monitoring through Log Analytics and KQL.

The most technically significant achievement was the Managed Identity implementation — the Function App reading from private Blob Storage with no credentials of any kind in the code. This is a pattern used extensively in production cloud environments because it eliminates the entire class of credential-related security vulnerabilities.

Key lessons learned from this project include:

- Identity is the most critical security layer in cloud environments — more so than firewalls or network controls
- The principle of least privilege must be applied at every layer: user accounts, service identities, and storage access
- Passwordless authentication through Managed Identities is not just more secure than password-based authentication — it is also simpler, because there are no credentials to manage, rotate, or protect
- Azure's tenant model provides strong isolation boundaries, but subscription and tenant management requires careful planning when operating under institutional restrictions
- Continuous logging and monitoring is not optional in a zero-trust architecture — the 'assume breach' principle requires that all activity be captured and queryable

Given more time and a premium license, this implementation would be extended with active Conditional Access policies and PIM just-in-time role elevation, completing the full zero-trust architecture described in the design phase. The foundation built in this project — correct tenant structure, least-privilege RBAC, Managed Identity, and log analytics — provides a solid base for those extensions.

7. References

- [1] Microsoft, "What is Microsoft Entra ID?," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/entra/fundamentals/whatis>
- [2] Microsoft, "What is Azure role-based access control (Azure RBAC)?," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/azure/role-based-access-control/overview>
- [3] Microsoft, "What are managed identities for Azure resources?," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/entra/identity/managed-identities-azure-resources/overview>
- [4] Microsoft, "What is Conditional Access?," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/entra/identity/conditional-access/overview>
- [5] Microsoft, "Azure Monitor Logs overview," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/azure/azure-monitor/logs/data-platform-logs>
- [6] Microsoft, "KQL quick reference," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/azure/data-explorer/kql-quick-reference>
- [7] Microsoft, "Zero Trust security," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/security/zero-trust/>
- [8] Microsoft, "Azure built-in roles," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/azure/role-based-access-control/built-in-roles>
- [9] A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN: Wiley, 2014.
- [10] Microsoft, "MFA can block over 99.9% of account compromise attacks," Microsoft Security Blog, 2019. [Online]. Available: <https://www.microsoft.com/en-us/security/blog/2019/08/20/one-simple-action-you-can-take-to-prevent-99-9-percent-of-account-attacks/>